

軟體工程導論

第一講：軟體工程導論與核心概念

授課教師：薛念林教授 (with Gemini AI)

逢甲大學 資訊工程學系

本章學習導覽與核心議題

- **1.1 技術演進脈絡：** 從機械化到普及化 AI
- **1.2 軟體工程起源與軟體危機：** 1968 NATO 會議與歷史重大失敗案例
- **1.3 軟體本質剖析：** 超越原始碼（程式、數據、SOP、文檔）
- **1.4 ISO 9126 品質模型：** 6 大特徵與關鍵子屬性實務案例
- **1.5 現代軟體多元樣貌：** Web、Mobile、ERP、嵌入式與 AI 平台
- **1.6 何謂軟體工程？：** 定義、工程流程 (SE Process)、約束與資源、4 大核心活動、工程要素與迷思
- **1.7 現代工具鏈與 AI 雙面刃：** CI/CD 流程與 AI 生命週期效益 / 風險矩陣
- **1.8 專業倫理與黑暗模式：** ACM/IEEE 守則與 UI/UX 黑暗模式黑白漫畫
- **1.9 常見問答與觀念總結**

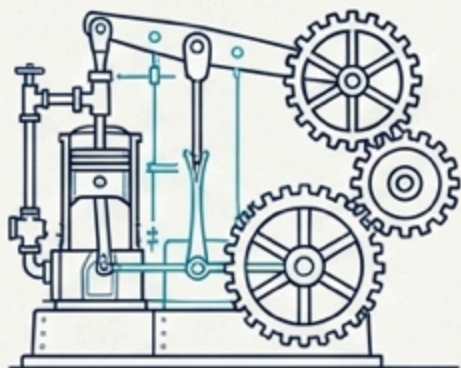
A systemic orientation to modern Software Engineering. Moving beyond ad-hoc programming to the rigorous, constrained, and ethical discipline of building the modern world's digital infrastructure.



The Evolution of Industrial Complexity

Industry 1.0 (Mechanization)

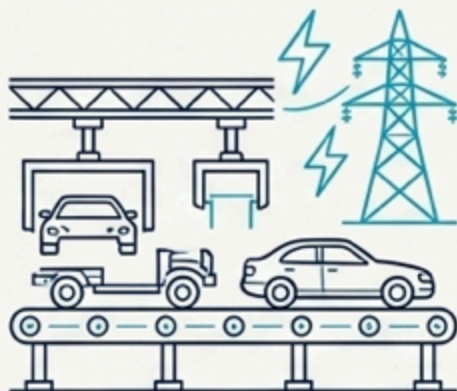
Late 18th-Mid 19th Century



Steam and water power.

Industry 2.0 (Mass Production)

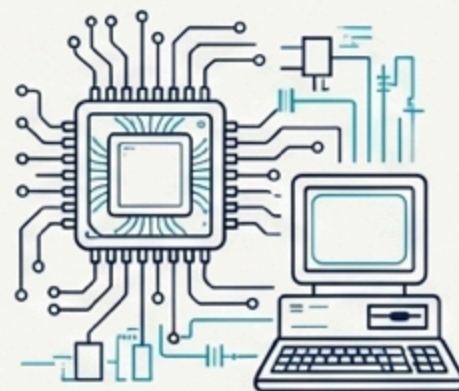
Late 19th-Early 20th Century



Electricity and assembly lines.

Industry 3.0 (Digital Revolution)

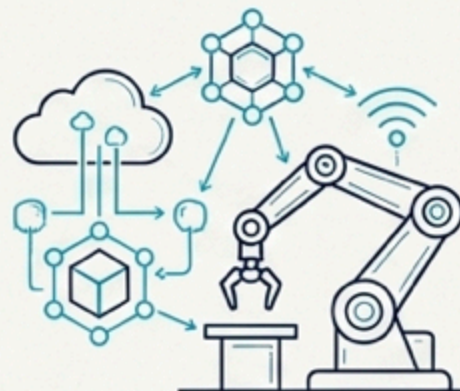
Mid-Late 20th Century



Computers, semiconductors, and PLCs.

Industry 4.0 (Cyber-Physical)

Present



IoT, AI, Cloud Computing, Advanced Robotics

Role of Software

1.0: Non-existent

2.0: Absent
(electro-mechanical)

3.0: Emergent
(Dawn of Software Engineering)

4.0: Pervasive
(Intelligent backbone)

1.1 四大工業革命時代

- **工業 1.0 —— 機械化時代（18 世紀末至 19 世紀中葉）：**
 - 蒸汽機與水力動力取代人體肌肉，工廠制度誕生。
 - **軟體角色：** 不存在，純粹物理機械範疇。
- **工業 2.0 —— 大規模量產時代（19 世紀末至 20 世紀初）：**
 - 電力、流水線裝配作業、標準化商品大規模製造。
 - **軟體角色：** 不存在，自動化純由硬體電路硬焊決定。
- **工業 3.0 —— 數位革命與軟體萌芽（20 世紀中葉至末期）：**
 - 半導體、微處理器、PLC 與個人電腦普及。
 - **軟體角色：** 正式獨立為專業學科，用於控制硬體與處理數據。
- **工業 4.0 —— 互聯、雲端與 AI 時代（21 世紀至今）：**
 - 虛實整合系統 (CPS)、物聯網、雲端運算與生成式 AI。
 - **軟體角色：** 成為現代文明運轉的基礎中樞神經。

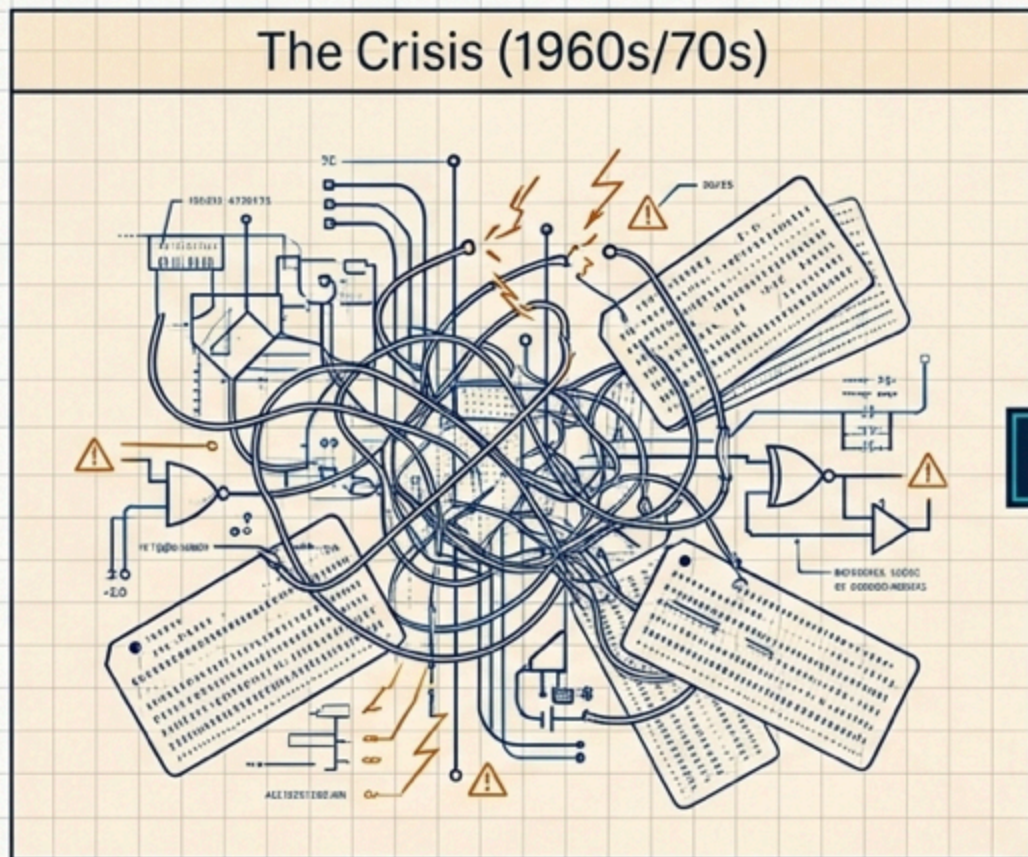
1.1 課堂互動活動：隨堂投票與討論

課堂投票與分組討論：

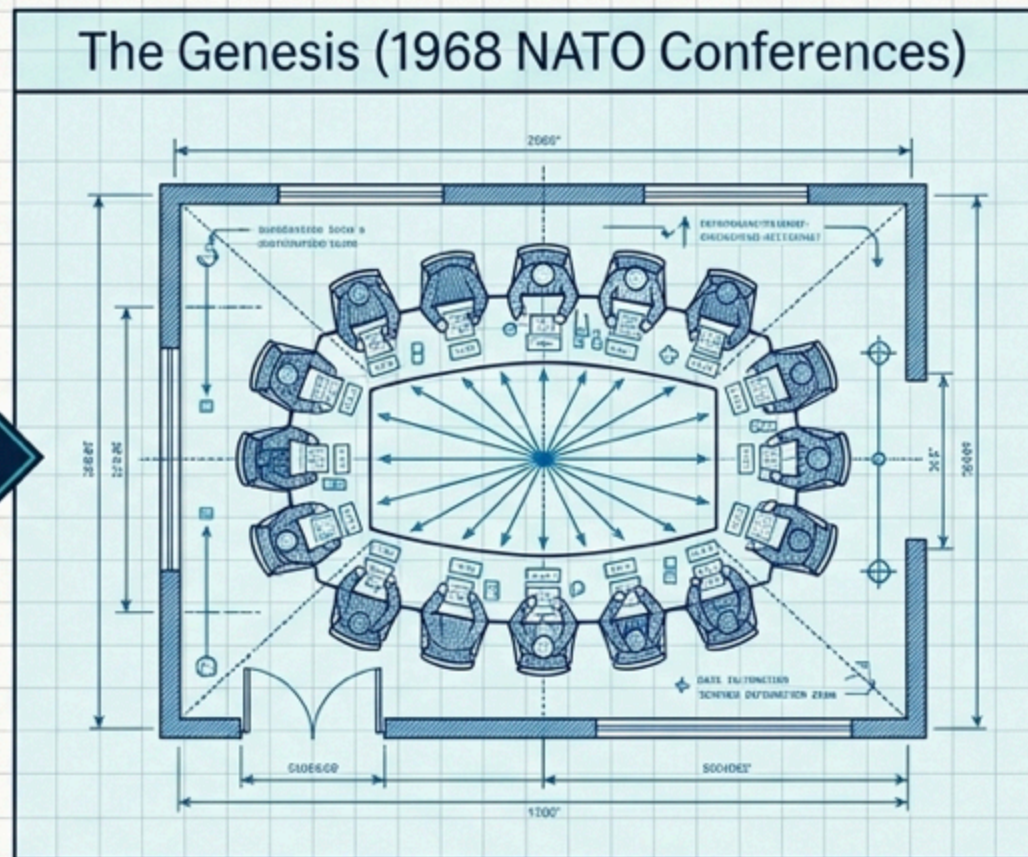
- **投票：** 檢視你日常使用的數位服務與裝置，有多少比例屬於工業 3.0 確定性邏輯，又有多少屬於工業 4.0 自適應 AI 聯網系統？
- **分組討論：** 請列舉一項你認為急需進行「工業 4.0 轉型」的傳統流程（如校園停車、醫院急診分流）。在軟體轉型過程中會面臨哪些工程挑戰？



Surviving the Software Crisis



Rapidly scaling complexity exposed the flaws of custom, ad-hoc programming. Projects consistently suffered from massive budget overruns, late deliveries, and outright system failures.



Experts gathered to establish a formal discipline—borrowing the predictability, safety margins, and systematic planning of physical engineering to manage digital complexity.

1.2 1960 年代後期的軟體危機

- 隨著硬體成本驟降，軟體需求與規模呈爆炸性增長。
- 危機典型徵兆：
 - 成本嚴重超支（經常為預算的 3~4 倍）。
 - 交付嚴重延宕，大量專案中途被迫放棄。
 - 缺陷頻傳、系統脆弱且完全無法維護。
- 核心根源：個人隨意編程的手工作法，完全無法應對跨團隊的大型複雜系統。

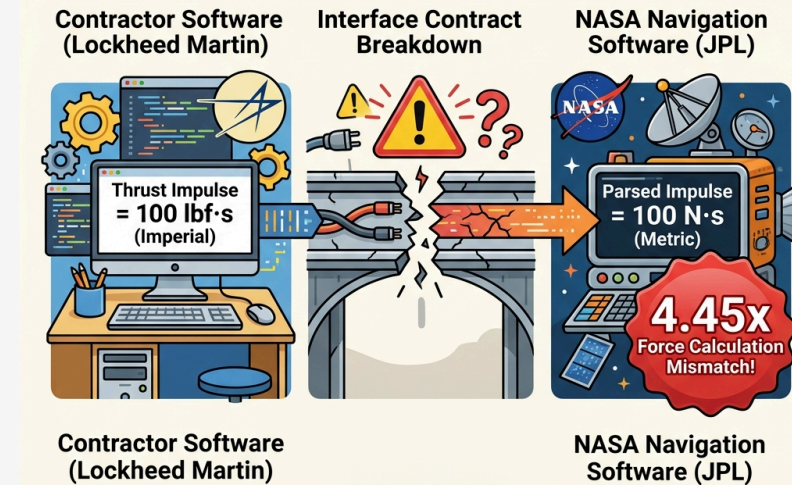


1.2 1968 年 NATO Garmisch 會議

- **1968 年 10 月於德國加米施 (Garmisch)：**
 - 50 位頂尖電腦科學家與業界主管齊聚一堂。
 - 正式確立並採用 「**軟體工程 (Software Engineering)**」 一詞。
- **學科使命：**
 - 推動軟體開發從個人手工藝，轉型為具備科學嚴謹度的工程學科。
 - 確立結構化方法論、形式化規格、成本估算、測試驗證與專案管理規範。

1.2 缺乏工程紀律的重大歷史代價

- 名古屋空難 (1994) :
 - 自動駕駛與機師手動操作產生人機介面 (HMI) 邏輯衝突，奪走 264 條生命。
- 火星氣候探測器 (1999) :
 - 價值 3.27 億美元的探測器因英制與公制力學單位未校驗而墜毀。
- 阿利安 5 號火箭 (1996) :
 - 64 位元浮點數轉 16 位元整數未做溢位例外處理，升空 37 秒引爆損失 3.7 億美元。



觀念檢驗題：軟體危機的本質 (CCQ 1)

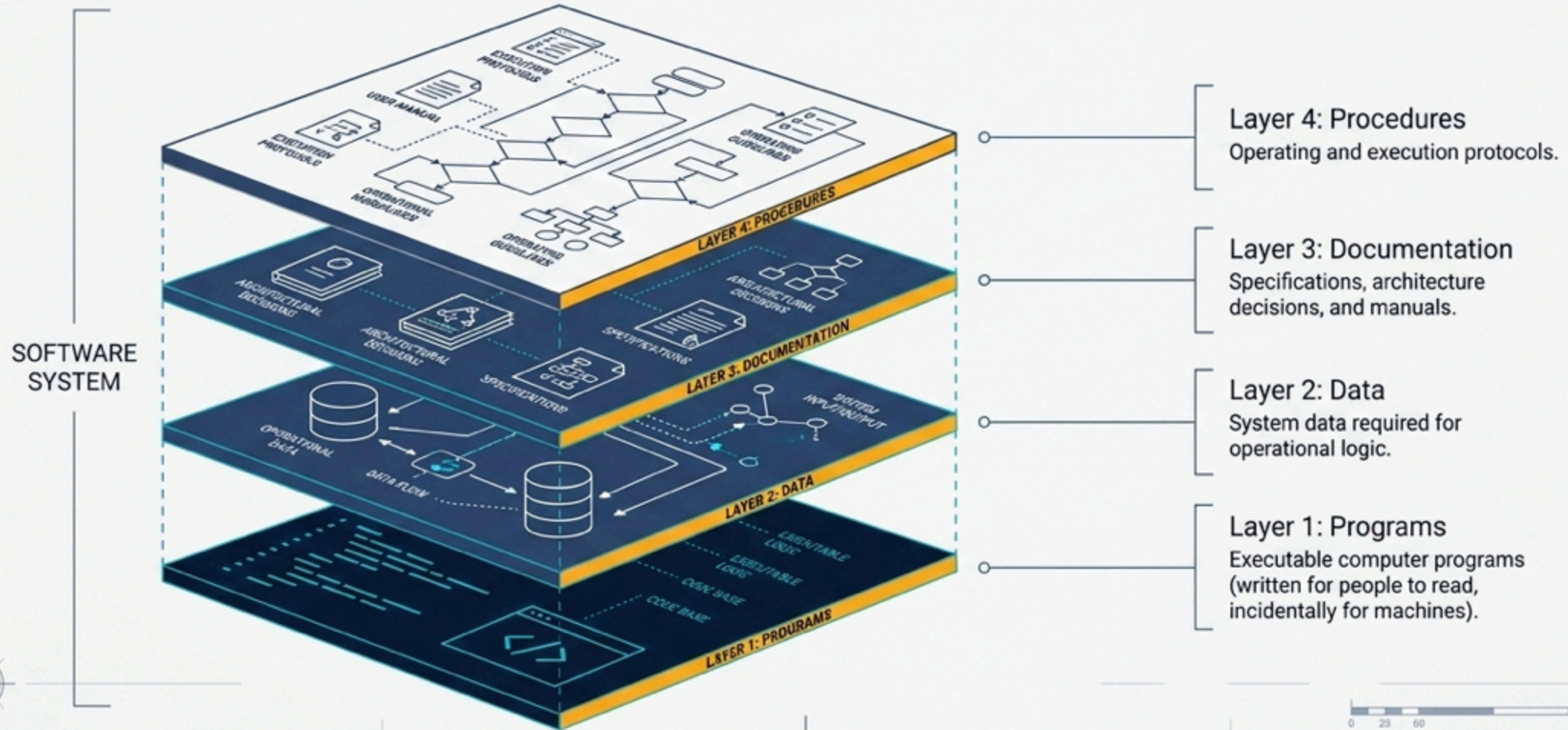
為何 1968 年的軟體危機無法僅透過採購運算速度更快的電腦或擴充記憶體來解決？

- **A.** 1960 年代後期硬體製造技術停滯不前。
- **B.** 危機本質上是人類心智面對複雜度、溝通成本與缺乏架構紀律的認知危機，更快的硬體只會放大問題規模。
- **C.** 當時的程式語言缺乏數學運算能力。
- **D.** 當時的硬體無法連接雲端架構。



The Anatomy of Software Exceeds Source Code

A failure in operational procedures is a failure of the software system. Physical hardware is explicitly excluded; software encompasses the intangible logic governing the system.



1.3 IEEE 對軟體的標準定義

軟體 (*IEEE 標準*)：與計算機系統運作相關的電腦程式、程序、以及可能伴隨的相關文件與資料。 ”

- **1. Programs (程式碼與二進制檔)**：Python, Java, C++, TypeScript 撰寫的執行邏輯。
- **2. Data & Schemas (資料與綱要)**：資料庫表格、設定檔、種子資料集與 AI 模型權重檔。
- **3. Operational Procedures (維運程序)**：部署腳本、備份例程、災難復原演練與 CI/CD 流程。
- **4. Documentation (架構文檔)**：架構決策記錄 (ADR)、OpenAPI 介面規格書、SRS 與使用手冊。

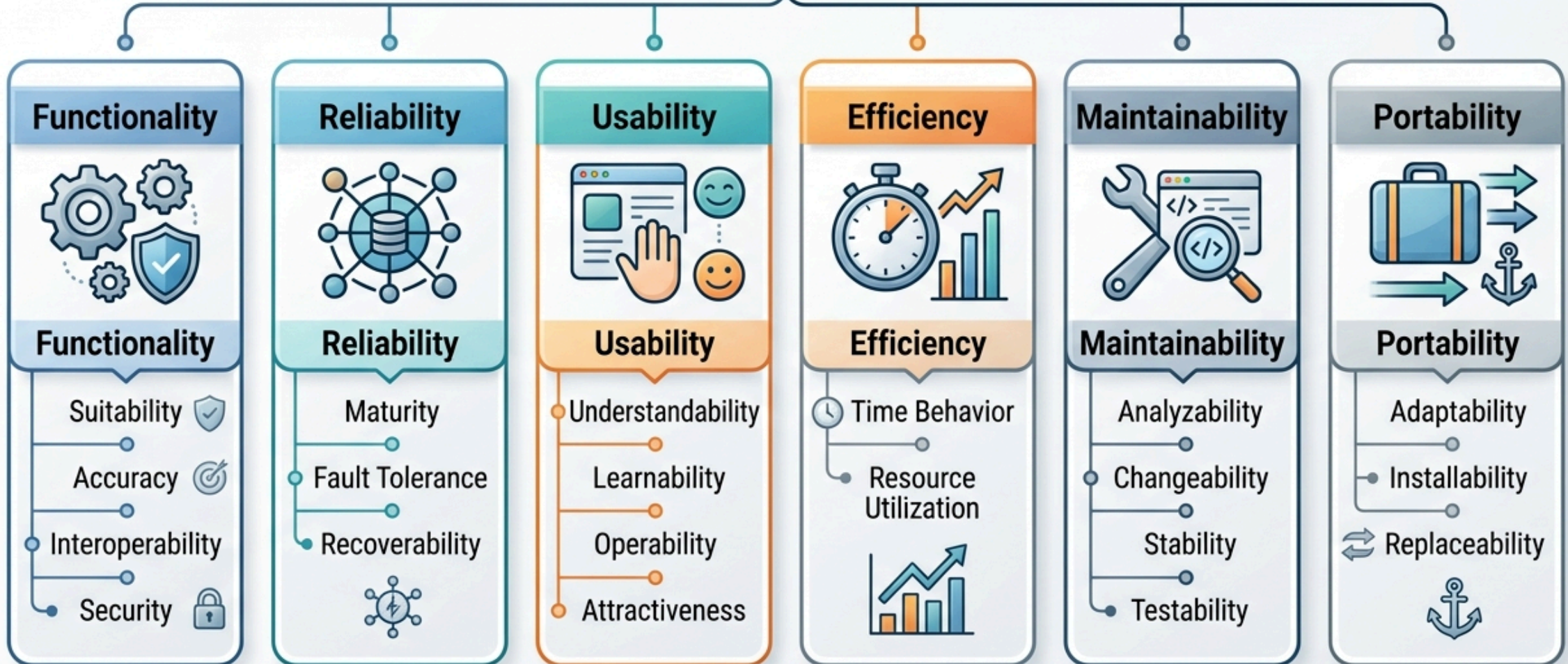
1.3 課堂互動活動：雙人分組討論

雙人討論：實務中的軟體冰山

- 挑選一項知名數位服務：**Google Maps**、**Uber** 或 **Spotify**。
- 兩人合作為四大支柱各列出一項具體產出物：
 1. 程式 (*Program*) | 2. 資料 (*Data*) | 3. 程序 (*Procedure*) | 4. 文檔 (*Documentation*)
- 反思：如果團隊遺失了所有部署程序與資料庫綱要，能否單憑原始程式碼在產線上順利重建業務？



ISO/IEC 9126 Software Quality Model



1.4 ISO 9126 六大特徵與關鍵子屬性

- **Functionality (功能性)**：適用性 (Suitability)、準確性 (Accuracy)、互操作性 (Interoperability)、資安性 (Security)。
- **Reliability (可靠性)**：成熟度 (Maturity)、容錯性 (Fault Tolerance)、可復原性 (Recoverability)。
- **Usability (易用性)**：易理解性 (Understandability)、易學習性 (Learnability)、易操作性 (Operability)、吸引力。
- **Efficiency (效率性)**：時間行為 (延遲 / 吞吐量)、資源利用率 (CPU / 記憶體)。
- **Maintainability (可維護性)**：可分析性 (Analyzability)、可變更性 (Changeability)、穩定性、可測試性 (Testability)。
- **Portability (可移植性)**：適應性 (Adaptability)、可安裝性 (Installability)、可替換性。

1.4 ISO 9126 子屬性實務案例

- **容錯性 (Fault Tolerance)**：主金流閘道逾時 → 系統自動重試備援金流，結帳不當機。
- **可復原性 (Recoverability)**：資料庫當機 → 30 秒內透過 Write-Ahead Log 復原交易一致性。
- **時間行為 (Time Behavior)**：搜尋 API 在 99% 的請求中延遲小於 80ms (p99 latency < 80ms)。
- **可測試性 (Testability)**：採用依賴注入 (DI)，讓單元測試能輕鬆 Mock 外部 API。
- **可安裝性 (Installability)**：透過 `docker compose up` 在 60 秒內啟動完整本機環境。

觀念檢驗題：軟體品質維度 (CCQ 2)

一個後端微服務採用依賴注入 (DI) 架構並建立結構化 JSON 日誌。當產線發生異常時，工程師在 5 分鐘內精準鎖定錯誤並安全修復。此案例展現了哪一項品質維度？

- **A.** 可移植性 (Portability)
- **B.** 可維護性（可分析性 Analyzability 與可變更性 Changeability）
- **C.** 易用性 (Usability)
- **D.** 功能合規性 (Functionality Compliance)



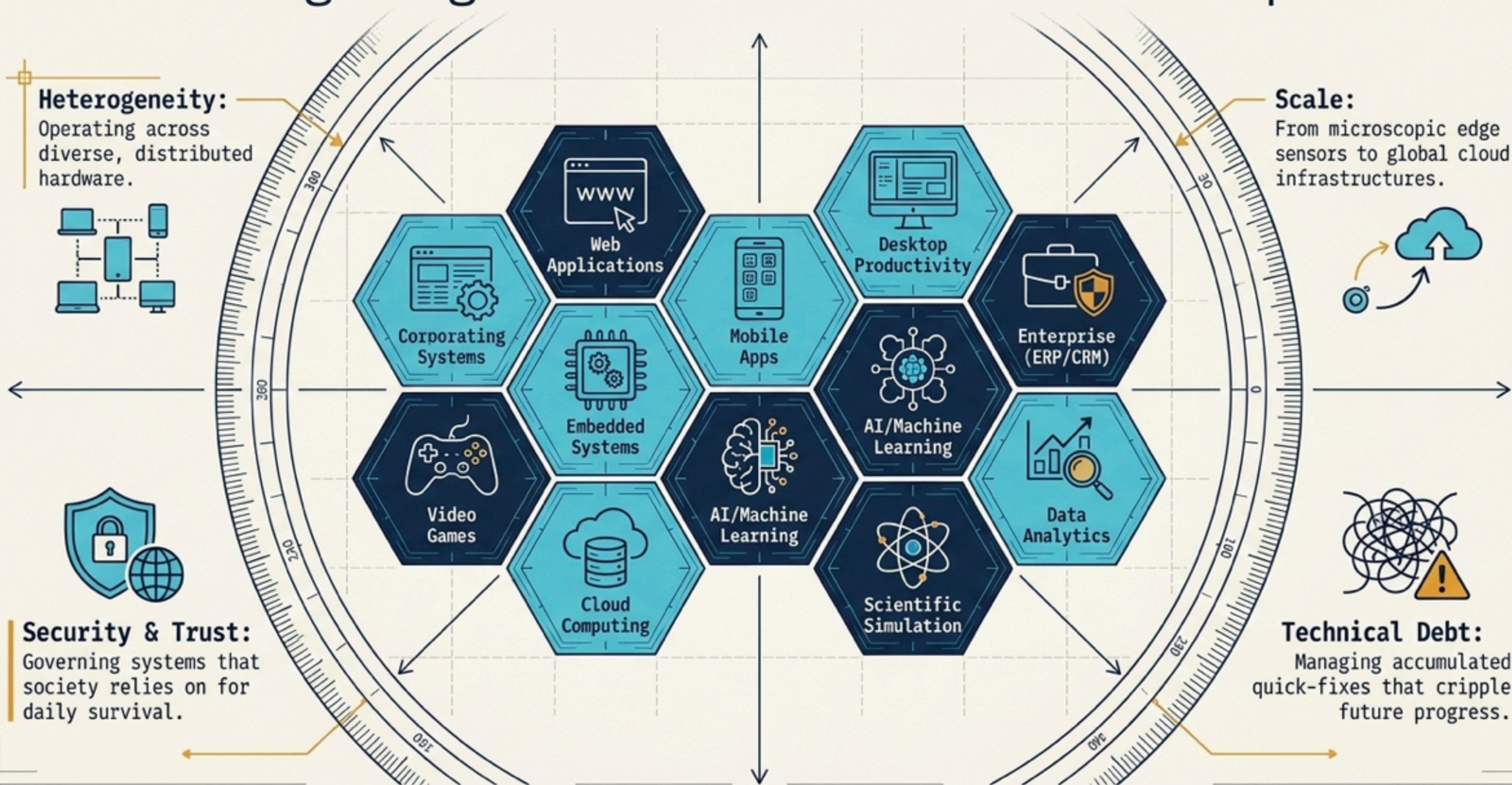
1.4 課堂互動活動：品質屬性權衡投票

品質屬性權衡投票與討論：

- 系統 A：加護病房 (ICU) 自動胰島素注射幫浦控制系統
- 系統 B：行動裝置熱門休閒手機小遊戲
- 投票：請為系統 A 與系統 B 分別選出最不可妥協的「前兩大」ISO 9126 品質屬性。
- 討論：為何在醫療設備中犧牲「容錯性」以換取「上市速度」是致命的，而休閒遊戲卻可適度接受非致命 Bug 以換取快速迭代？



Navigating the Modern Software Landscape



1.5 主要軟體應用領域

- **Web 與 SaaS 雲端平台**：彈性架構、高可用性、微服務、零停機部署（Slack、Netflix）。
- **行動應用程式 (Mobile Apps)**：電池受限、觸控互動、離線同步（iOS/Android 原生與跨平台）。
- **企業級核心系統 (ERP / CRM)**：嚴格 ACID 資料庫交易、工作流管理（SAP、Salesforce）。
- **嵌入式韌體與物聯網 (IoT)**：嚴苛即時性、極限記憶體、零容錯航電與車載韌體。
- **AI 與機器學習平台**：資料管線、向量資料庫、GPU 運算加速與大語言模型推理。
- **科學運算與 CAD**：高精度浮點數運算、物理模擬與硬體加速。

1.5 課堂互動活動：系統分類與混合架構

系統分類與混合架構探討：

- 剖析現代 **Tesla** 自駕智慧電動車 涵蓋了哪些軟體範疇？
 - 煞車與動力控制的即時嵌入式韌體。
 - 中控觸控導航與娛樂系統 UI。
 - 車隊遙測與 OTA 遠端更新的雲端原生微服務。
 - 邊緣端 FSD 全自動輔助駕駛 AI 模型。
- 討論：為何各子系統的發布與驗證週期具有巨大差異？



CODING



SOFTWARE ENGINEERING

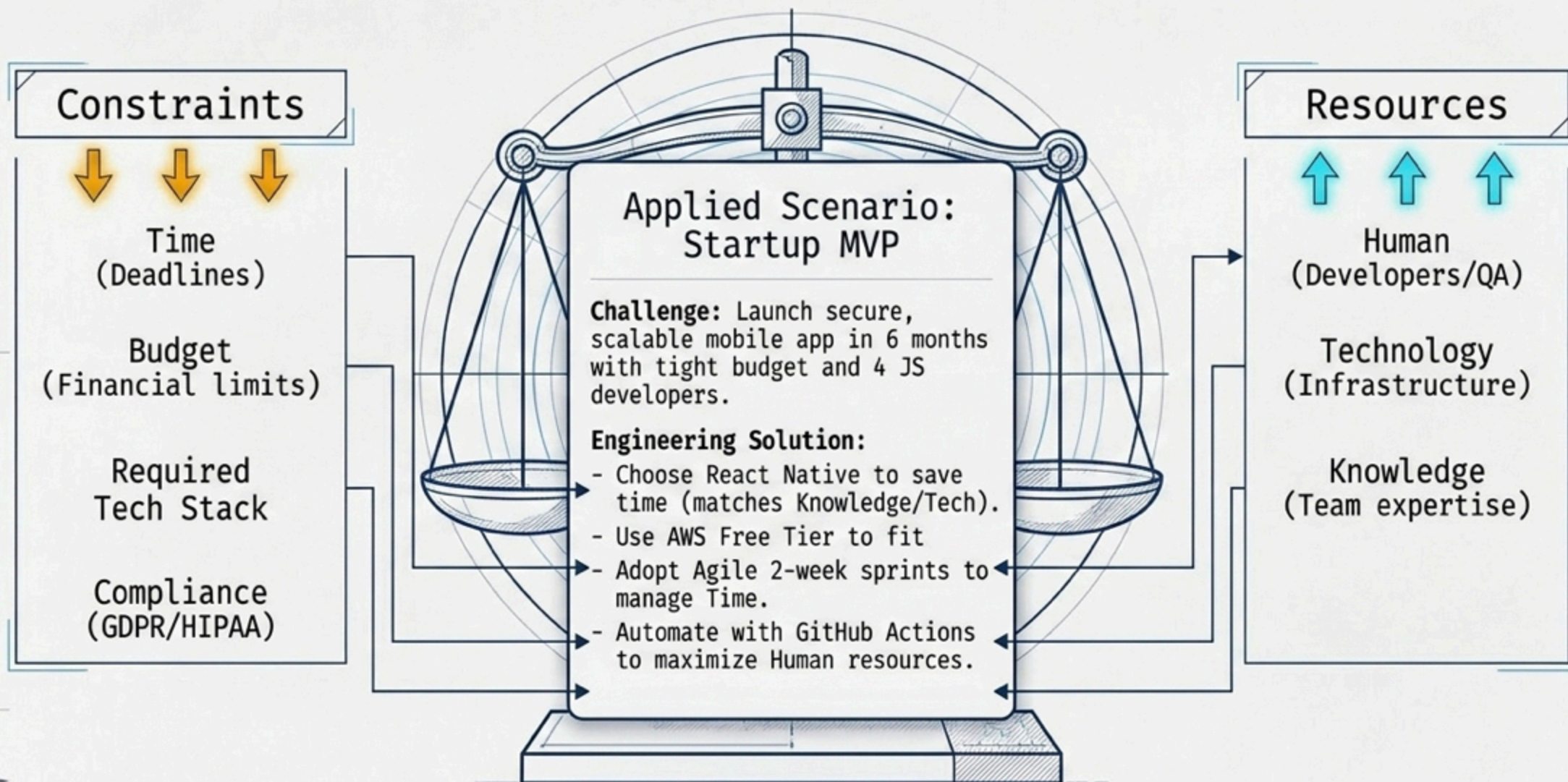


1.6 何謂軟體工程？

軟體工程 (Software Engineering)： 是一門工程學科，關注軟體生產的所有層面——從前期的系統需求規格定義，到系統上線後的維護與長期演進。

- **1. 工程學科（在約束與資源下工作）：** 工程運用科學嚴謹度與啟發準則，在嚴苛的約束下善用有限資源解決實際問題。
- **2. 軟體生產的所有層面（軟體工程流程 SE Process）：** 由一套系統化的流程組織活動、方法、角色與產出物，確保系統全生命週期的可控性。

The Engineering Balancing Act



1.6 工程方程式：約束與資源的平衡

- **Constraints (現實約束)：**

- **時間 (Time)：** 專案死線、市場搶先發布窗口。
- **預算 (Budget)：** 薪資上限、雲端費用配額、第三方軟體授權費。
- **技術與平台 (Tech)：** 遺留資料庫相容性、手機 OS 版本限制。
- **法規 (Regulations)：** GDPR 個資隱私法、HIPAA 醫療法規、PCI-DSS。

- **Resources (可用資源)：**

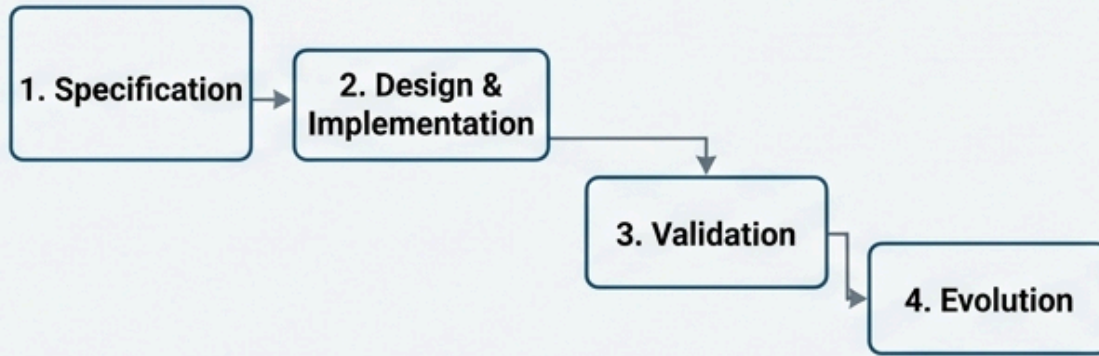
- **人力 (Human)：** 開發者技術專長、UI/UX 設計師、QA 測試員。
- **工具與基建 (Tools)：** 雲端運算配額、CI/CD 自動化、開源函式庫。
- **領域知識 (Knowledge)：** 業務邏輯理解度與設計模式掌握度。

1.6 實務案例：醫療新創團隊 MVP 突圍戰

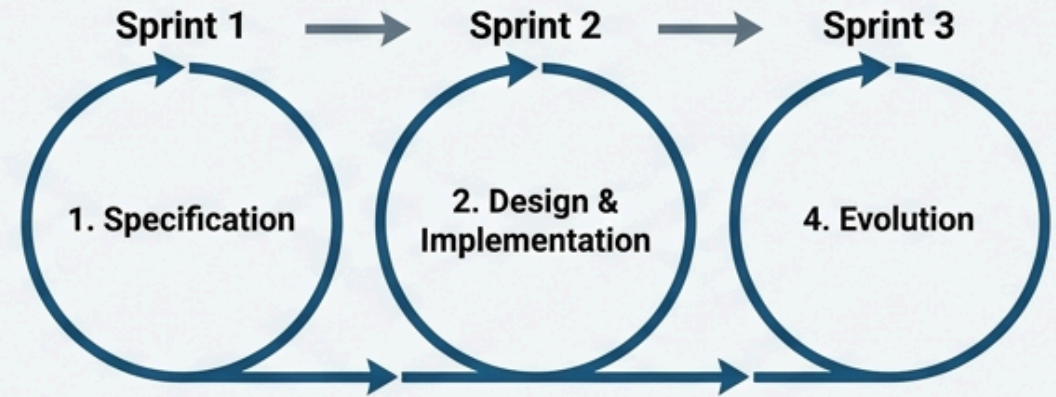
- 情境：在 6 個月內以 8 萬美元有限預算推出符合 HIPAA 法規的遠距醫療問診 App。
- 工程權衡解決方案：
 1. 範疇精簡：先打造具備視訊問診與處方預約的 MVP，推遲複雜保險理賠。
 2. 跨平台開發：使用 **Flutter** 建立單一代碼庫發布雙平台，節省 40% 人力。
 3. 託管雲端服務：採用符合 HIPAA 的託管 BaaS (Supabase/Firebase) 替代自建伺服器。
 4. 全自動化 CI/CD：**GitHub Actions** 自動執行 Lint 與測試，維持 3 人小團隊的高代碼品質。

Universal Core Activities Across Different Process Models

Waterfall Model

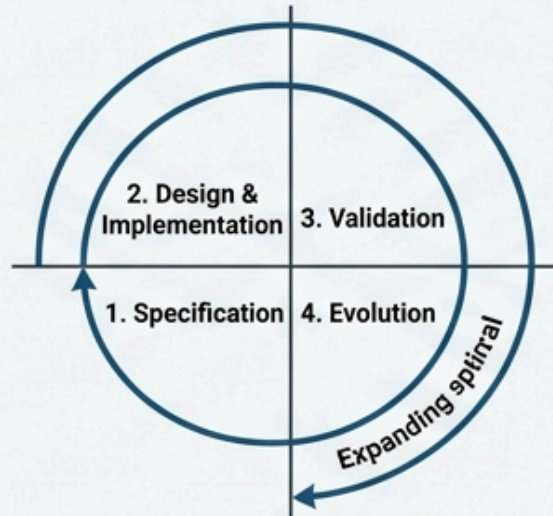


Agile / Iterative Model

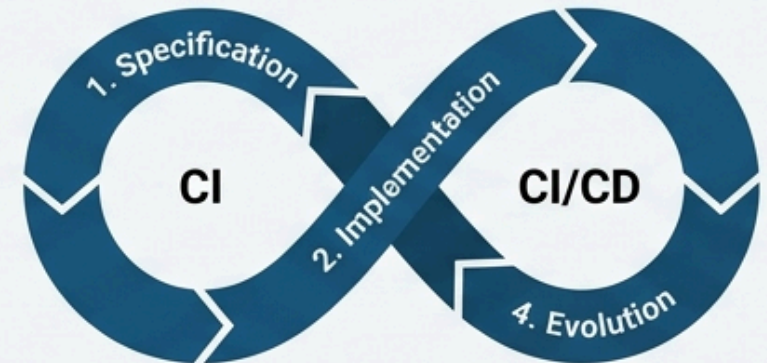


Spiral Model

Spiral Model



Continuous DevOps



1.6 軟體工程流程四大核心活動

核心活動	關鍵產出物 (Artifacts)	若忽視此活動的嚴重代價
1. 需求規格 (Specification)	使用者故事、SRS 規格書、驗收條件	打造出完全錯誤的產品；重工成本高達 100 倍
2. 設計與實作 (Design & Impl)	系統架構 ADD、資料庫 ERD、原始碼	義大利麵混亂代碼、無法擴展、累積巨額技術債
3. 驗證與確認 (Validation V&V)	自動化測試套件、CI 報告、缺陷清單	產線重大當機事故、資料遺失、資安漏洞爆發
4. 系統演進 (Evolution)	發布說明、資料庫 Migration 腳本	軟體腐化、相依套件過期漏洞、系統被迫報廢

1.6 課堂互動活動：情境分析

情境分析：哪一項核心活動失敗了？

- **情境：** 一個技術團隊耗時 6 個月，為社區生鮮外送 App 打造了極致流暢、零 Bug 的加密貨幣結帳功能。該代碼通過了 100% 的單元測試且產線上零當機。然而上線後使用率為 0%，因為社區長輩只習慣使用「貨到付款」。
- **問題：** 四項核心活動中，哪一項發生了根本性失敗？
- **核心啟示：** 為何 100% 的代碼測試覆蓋率完全無法挽救需求規格 (Specification) 的失敗？



• CORE ELEMENTS OF SOFTWARE ENGINEERING •



Disciplines (Best Practices)



- Spec before design
- Interface-based



Principles (Core Foundations)



- Abstraction
- Anticipation of Change

1.6 軟體工程所涵蓋的各項要素

- **工程實務紀律 (Disciplines)**：先規格後設計、先設計後編碼、基於介面開發、變更管理、ADR 架構決策記錄、代碼審查...
- **核心設計原則 (Principles)**：抽象化、模組化、預期變更、開閉原則 (OCP)、KISS、最小驚訝原則 (POLA)...
- **流程方法論 (Methods)**：瀑布模型、敏捷 / Scrum、螺旋模型、測試驅動開發 (TDD)、CI/CD & DevOps...
- **實務啟發準則 (Heuristics)**：尼爾森 10 大易用性準則、SOLID 物件導向原則、Clean Code、DRY、YAGNI...

1.6 破除常見的軟體開發迷思

- 迷思 1：「專案進度落後了，只要多加幾位工程師就能趕上。」
 - 現實（布魯克斯法則）：為落後的專案增加人手只會讓進度更落後（溝通通道呈 $O(n^2)$ 增長）。
- 迷思 2：「軟體具有數位彈性，後期變更需求成本很低。」
 - 現實：後期修改核心架構的成本可高達初期的 100 倍。
- 迷思 3：「把專案外包後，我們就不需要技術管理了。」
 - 現實：外包若缺乏架構治理，交付的系統往往充斥技術債。

Myth: Changing requirements late is cheap!

Requirement change early during blueprint design (Cost: 1x)



Requirement change at the very end of construction (Cost: 100x)



觀念檢驗題：布魯克斯法則 (CCQ 3)

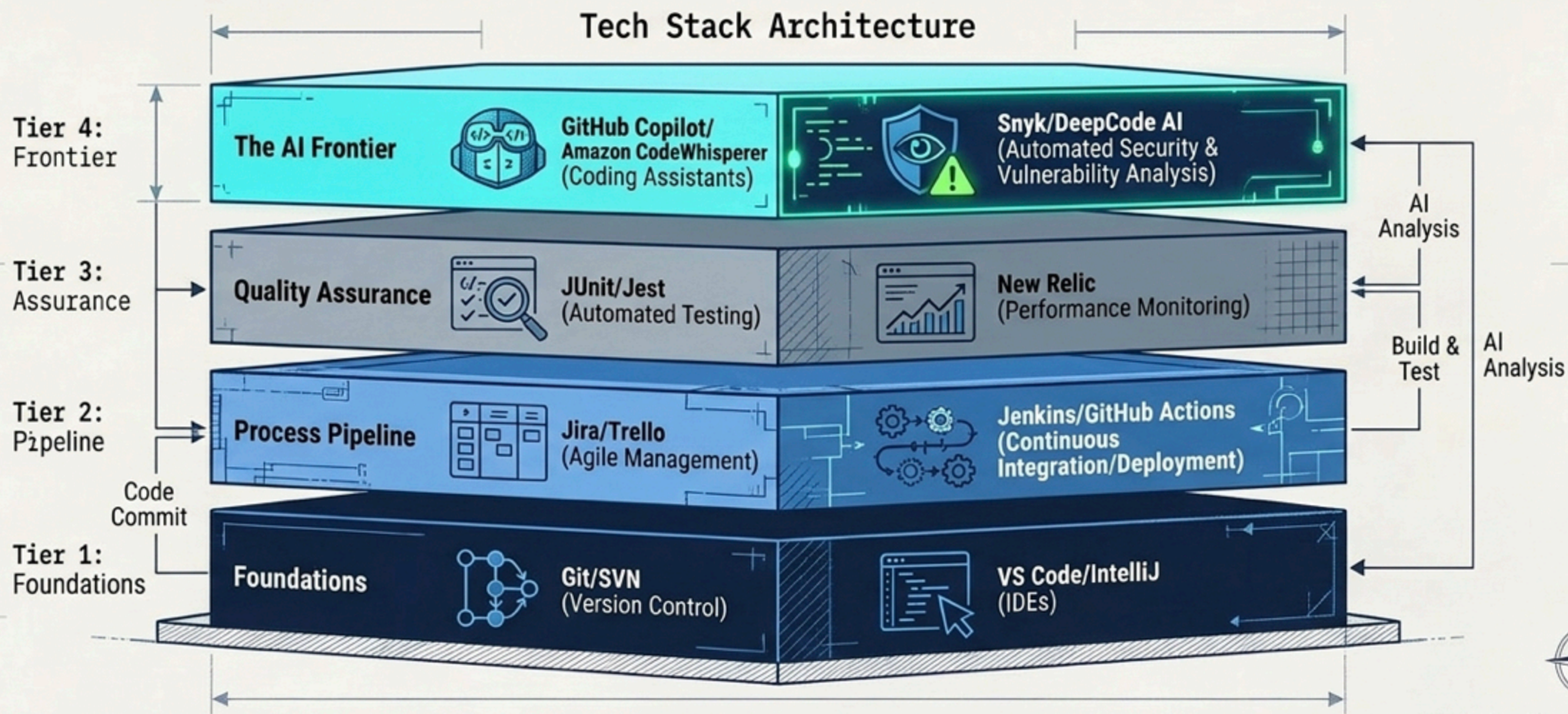
專案目前落後 3 週且離上線只剩 2 週，專案經理決定立即招聘 4 位新進工程師加快進度。依據布魯克斯法則，最可能的結果為何？

- **A.** 專案將提早 1 週完成。
- **B.** 專案將更為延宕，因為資深成員必須停下手邊工作去培訓新成員，且溝通成本劇增。
- **C.** 現有工程師的寫代碼速度將翻倍。
- **D.** 團隊整體的溝通複雜度保持不變。



Automating Discipline: The Modern Toolchain

Professional engineering requires strict discipline (code reviews, test coverage, version control). Modern tooling automates these mandates.



1.7 現代軟體工程核心工具鏈

- **版本控制系統 (Git)**：分支管理、Pull Request 協同審查與變更追蹤。
- **現代開發環境 (IDEs)**：VS Code、IntelliJ 提供即時靜態分析、智慧重構與除錯。
- **CI/CD 自動化管線**：GitHub Actions 於每次 Commit 自動編譯、測試、資安掃描與部署。
- **自動化測試套件**：單元測試 (PyTest/Jest)、整合測試與 E2E 測試 (Playwright)。
- **可觀測性與監控**：Datadog、Prometheus、Sentry 捕獲即時遙測與產線 Error Trace。
- **AI 編程助理**：GitHub Copilot、Cursor、Gemini AI 提供智慧配對編程。

1.7 AI 在軟體生命週期的效益與風險矩陣

生命週期階段	AI 帶來的核心效益	潛在風險與技術陷阱
1. 需求分析	快速草擬使用者故事、整理 Gherkin 驗收條件	憑空幻覺 (Hallucination) 出虛構業務規則；遺漏隱性組織文化
2. 架構設計	推薦合適架構模式、自動生成 UML 與 ERD	過度工程化 (Over-engineering)；忽略網路延遲與資安隔離 SLA
3. 程式編程	自動完成樣板代碼、加速演算法撰寫、減少疲勞	「盲目相信代碼 (Vibe Coding)」陷阱；引入隱蔽資安漏洞與授權污染
4. 測試驗證	自動生成邊界測試數據、合成測試案例	「回音室測試 (Echo-chamber)」(測試只驗證了 AI 寫出的錯誤邏輯)
5. 維護演進	解讀老舊無文件代碼、自動生成 API 規格書	自動重構引發無感行為退化 (Regression)；累積不易察覺技術債

1.7 課堂互動活動：Vibe Coding 大挑戰

隨堂投票與討論：AI 實務應用反思

- **投票：** 使用 GitHub Copilot 或 ChatGPT 產生代碼時，你在按 Commit 前有多常逐行審查並完全理解每行邏輯？
(1: 總是逐行理解 | 2: 大部分時候 | 3: 很少 | 4: 從不)
- **討論：** 假設 AI 產生了一段 200 行複雜非同步資料庫操作代碼，且順利通過了 2 個基本測試。這段代碼可以直接推向產線嗎？專業工程師必須補足哪些驗證紀律？



1. The Public Interest:
Software engineers must prioritize the safety, health, and welfare of the public above all other obligations.

Client and Employer:
Act in their best interests (subordinate to public safety).

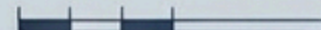
Product:
Ensure products meet the highest professional standards.

Judgment:
Maintain total integrity and independence in technical decisions.

Management:
Promote ethical management and project practices.

Profession:
Advance the integrity and reputation of the engineering discipline.

Colleagues and Self:
Be fair and supportive; participate in lifelong learning.



1.8 ACM/IEEE 軟體工程倫理守則：八大原則

1. **Public**（公眾利益）：將公眾的健康、安全與福祉置於首位。
2. **Client & Employer**（客戶與雇主）：在符合公眾利益前提下維護雇主利益。
3. **Product**（產品品質）：確保軟體產品與維護符合最高專業標準。
4. **Judgment**（獨立判斷）：在技術評估中保持誠信與獨立性。
5. **Management**（工程管理）：推廣倫理管理實務，給予合理估算與健康環境。
6. **Profession**（專業聲譽）：維護並增進軟體工程專業的誠信與聲譽。
7. **Colleagues**（同儕互助）：公平對待同儕，建立良性指導與回饋。
8. **Self**（自我提升）：終身學習，精進專業技能並推廣倫理實務。

When Engineering Fails: The Cost of Ethical Breach

Incident Report

Malicious Design: Volkswagen's Dieselgate

Software explicitly engineered to detect testing environments and artificially cheat emission standards, betraying public health and environmental regulations.

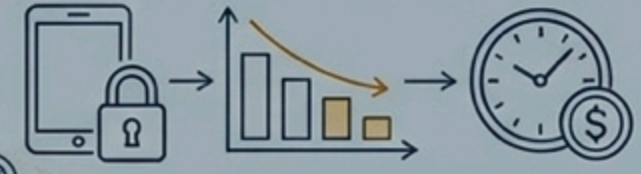


The diagram shows a car engine with a gauge labeled 'cheat' and a graph of 'Emissions' vs 'Emission Levels'. The graph shows a curve that rises sharply after a certain point, indicating a cheat. A warning sign is also present.

Incident Report

Engineered Exploitation: Planned Obsolescence

Designing software to artificially expire or restrict user control through locked ecosystems and restrictive licensing, prioritizing corporate profit over the Product and Client pillars.



The diagram shows a smartphone with a lock, a bar chart with a downward arrow, a clock, and a dollar sign. A large orange stamp reads 'WARNING: ETHICAL BREACH DETECTED'.

Technical callioion
ecoazonars that releases
entastocetun wësted
eacameeting xör teur
cameneetnad selutions.

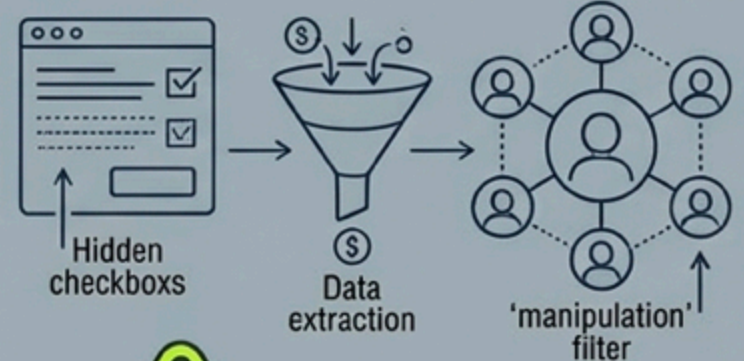


Tutrm an engimers will in
extracte the malicious ethical
architectuners for
unauthorized targeting.

Incident Report

Psychological Manipulation: Dark UX & Cambridge Analytica

Deceptive user interfaces designed to trick users into unconsented actions (e.g., forced subscriptions) and the malicious misuse of data architectures for unauthorized targeting.



The diagram shows a user interface with 'Hidden checkboxes', a funnel for 'Data extraction', and a 'manipulation' filter. A warning sign is also present.

WARNING: ETHICAL BREACH DETECTED

1.8 重大工程倫理失守事件

- 福斯汽車「柴油門」事件 (2015)：

- 工程師依指示撰寫作弊軟體，在檢驗狀態下掩飾高達法定 40 倍的 toxic NO_x 廢氣排放。
- 引發數百億美元天價罰款、刑事定罪與嚴重的環境公害。

- 劍橋分析事件 (2018)：

- 不當收集數千萬社群用戶個資進行政治操控，凸顯隱私設計的重要。

- 計畫性報廢 (Planned Obsolescence)：

- 透過軟體更新蓄意降低舊款硬體效能，迫使消費者換機。

DECEPTIVE UI/UX DARK PATTERNS



1.8 欺騙性「黑暗模式」(Dark Patterns) 解析

- **1. Roach Motel (蟑螂屋 / 訂閱迷宮) :**
 - 1 鍵快速訂閱容易，取消訂閱卻隱藏在層層選單或強迫打電話。
- **2. Confirmshaming (羞辱性確認) :**
 - 拒絕按鈕使用情緒勒索文案 (「不要，我討厭省錢與小狗」)。
- **3. Hidden Costs & Sneak into Basket (偷塞購物車) :**
 - 結帳最後一步自動預先勾選附加保險或偷加手續費。
- **4. Fake Urgency (製造虛假緊迫感) :**
 - 虛假倒數計時器 (「優惠只剩 5 秒鐘！」) 逼迫衝動下單。

觀念檢驗題：工程倫理判斷 (CCQ 4)

依據 ACM/IEEE 倫理守則，若雇主指示工程師實作一段會偽造安全合規檢驗報告的作弊演算法，工程師的倫理義務為何？

- **A.** 順從照做，因為雇主支付工程師薪資。
- **B.** 拒絕實作並向上回報，因為「公眾利益」永遠高於對雇主的忠誠。
- **C.** 照常實作代碼，但刻意不寫文檔。
- **D.** 將代碼外包給第三方廠商以規避法律責任。



1.8 課堂互動活動：黑暗模式偵探

黑暗模式偵探活動：

- **調查：** 回想你最近在 App 或網站上遇過的一個欺騙性黑暗模式。
- **剖析：** 該設計違反了 ACM/IEEE 倫理守則中的哪一項原則（公眾利益、產品品質、或獨立判斷）？
- **重構：** 你會如何重新設計該流程，使其既能兼顧合理的商業轉化，又能保持完全的誠信與透明？



The Mindset of a Software Engineer

Master Architect's Sign-Off

SIGNATURE BLOCK 1: SYSTEM HOLISM		APPROVAL STAMP
Beyond the Code: Software is not just algorithms; it is a complete system of programs, procedures, documentation, and data.		
SIGNATURE:	DATE:	ENGINEER'S SEAL



STATUS:
VERIFIED

INSPECTOR:
CHIEF ARCHITECT

SIGNATURE BLOCK 2: QUALITY ASSURANCE		APPROVAL STAMP
Beyond "Working": True quality demands usability, dependability, efficiency, and maintainability—not just conforming to basic requirements.		
SIGNATURE:	DATE:	ENGINEER'S SEAL



STATUS:
COMPLIANT

INSPECTOR:
QA LEAD

SIGNATURE BLOCK 3: PRACTICAL REALITIES		APPROVAL STAMP
Beyond Theory: Unlike theoretical computer science, Software Engineering focuses on the practical development of useful systems under strict real-world constraints (time, budget, human capital).		
SIGNATURE:	DATE:	ENGINEER'S SEAL



STATUS:
VALIDATED

INSPECTOR:
PROJECT MANAGER

SIGNATURE BLOCK 4: ETHICAL OBLIGATION		APPROVAL STAMP
Beyond the Screen: The ultimate metric of a software engineer's success is the protection and advancement of the public interest.		
SIGNATURE:	DATE:	ENGINEER'S SEAL



STATUS:
APPROVED

INSPECTOR:
ETHICS OFFICER



1.8 專業心態：數據、開發者與使用者

- 開發者與使用者的糾葛：
 - 軟體絕非孤立的代碼——它直接牽動真實人類的工作流程、生計與安全。
- **Data is Gold（數據即黃金）：**
 - 數據真實性、隱私治理與演算法公平性是現代工程師的核心責任。



數位小鎮的秘密

1.9 軟體工程常見問答 (FAQ)

- **Q：電腦科學 (CS) vs. 軟體工程 (SE)？**

- CS：數學與計算理論基礎（演算法、資料結構、複雜度）。
- SE：實務應用工程——在時間、預算與現實約束下打造可靠軟體。

- **Q：軟體成本主要花在哪裡？**

- 初始開發：約 60% 開發，40% 驗證測試。
- 完整生命週期：上線後的維護與演進佔據總成本的 **70%～80%**。

- **Q：是否存在放之四海皆準的「最佳方法論」？**

- 不存在。關鍵在於根據領域約束與商業目標為特定問題挑選最適當的工具與流程。

本章觀念總結：填空挑戰

檢驗你對第 1 章核心觀念的掌握程度：

1. 依據 IEEE 定義，軟體包含程式、數據、維運程序以及 [_____]。
2. 為落後的軟體專案增加人手只會讓進度更為延宕，此現象稱為 [_____] 法則。
3. [_____] 品質模型定義了包含功能性、可靠性、易用性、效率性、可維護性與可移植性 6 大維度。
4. ACM/IEEE 軟體工程倫理守則的第一原則優先保障 [_____] 利益。



參考文獻與延伸閱讀

- Sommerville, Ian. *Software Engineering* (10th Edition). Pearson. [官方網站](#)
- Brooks, Frederick P. *人月神話 (The Mythical Man-Month)*.
- ISO/IEC 9126-1:2001. *Software engineering — Product quality — Part 1: Quality model*.
- ACM/IEEE 軟體工程倫理與專業實務守則. [IEEE CS 官網](#)
- Brignull, Harry. *Deceptive Patterns: Exposing the Tricks Tech Companies Use to Control You*.